

2025 年中国研究生数学建模竞赛

A 题

通用神经网络处理器下的核内调度

基于驻留段约束的缓存与流水协同优化

历史真题研究阅读版

本稿用于学习研究，不是正式参赛提交版。

正文采用 12 pt 字号与约 18 pt 基线间距，便于阅读与复核。

基于驻留段约束的缓存与流水协同优化

摘 要

针对通用神经网络处理器中拓扑依赖、有限连续缓存与多执行单元并行的耦合，建立以生命周期收缩、驻留段分配和固定驻留图重排为核心的调度方法。在六个题面计算图上，分别求解最小驻留、低额外搬运和搬运预算下的执行时间优化。下述六图结果依次对应卷积小图、大图，注意力小图、大图及矩阵乘小图、大图。

对于问题一，以缓冲区活跃区间建立峰值目标，证明在申请为根、释放为叶的条件下，固定操作顺序的生命期收缩不增加峰值；再比较编号、窗口贪心与深度优先等八种顺序。卷积小图峰值由 7560 降至 6984，其余五图分别为 14048、4884、7972、9728、35328。同时给出操作必需容量下界，明确候选结果尚无全局最优保证。

对于问题二，用四状态描述逻辑生命与物理驻留，结合连续区间最佳适配及未来使用距离选择换出对象；受保护对象形成碎片时，以显式换出和重装恢复空间，并计入全部代价。六图额外搬运量依次为 44766、75546、3620、32852、34944、460800，卷积小图相对编号基准降低 48.90%。同顺序的换出准则对照表明，考虑单次搬运代价并不普遍优于单纯未来距离。

对于问题三，在搬运量零增量约束下比较地址轮换，并在含地址复用依赖的固定驻留图上实施列表重排。证明该重排保留地址互斥及换出集合，最终时间分别为 465892、1044075、37744、150499、173274、1749773 周期，相对问题二降低 1.30% 至 33.38%。注意力大图的向量单元忙碌比例由 46.46% 提高至 69.74%。进一步区分原始工作下界与固定驻留下界，六图相对后者的差距为 1.53% 至 24.95%，仅用于评价保留当前驻留关系后的改进空间。

所有方案经原始依赖、连续地址、使用时驻留和执行单元互斥重算。所得改善针对题面抽象模型及本文的换出期间地址复用解释，尚未由官方评价器或真实芯片测量确认。

关键词：有向无环图；生命周期；连续地址分配；换入换出；流水调度

1. 问题重述

题目将算子细化为由原子指令构成的有向无环图，节点包括指定执行单元上的操作，以及缓冲区申请和释放操作 [1]。每个缓冲区只属于一种缓存，需占据长度确定的连续地址区间。不同执行单元可以并行工作，同一执行单元只能串行执行。数据依赖和物理地址复用都会引入等待，因此拓扑顺序、缓存位置与实际完成时间必须联合考虑。

问题一要求在不考虑具体缓存容量限制时，寻找最大累计驻留量尽可能小的拓扑序。问题二在给定各类缓存容量后，为缓冲区确定偏移，必要时加入换出及换入操作，使额外搬运量尽可能小。问题三则允许进一步调整顺序和分配，在搬运量不显著增加的前提下降低执行时间。三个问题分别突出存储压力、物理可行性和执行并行性，目标之间存在竞争。

本文对六个输入图使用相同算法组合，不以算子名称编写专用调度规则。题面未给出“不显著增加”的数值阈值，因此问题三采用更严格的搬运量零增加约束。研究重点是获得可核验的完整调度方案，并明确启发式与评价模型的适用边界。

2. 数据理解与总体分析

2.1 输入结构与审计

输入同时提供节点和边的 JSON、CSV 表示。节点编号用于识别对象，不代表最优执行次序。操作节点的缓冲区列表同时包含输入与输出，不能仅依据列表顺序推断读写属性。本文直接使用题面给出的依赖边约束生产与消费先后。

六图规模如下。读取时逐字段比对两种表示，检查节点编号、重复边、缓存类型、大小及申请释放配对。全部输入通过检查，没有对缺失值进行填补，也没有删除任何节点或依赖。

表 1: 输入计算图规模

计算图	节点数	依赖边数
Conv_Case0	2580	3869
Conv_Case1	36086	85653
FlashAttention_Case0	1716	2712
FlashAttention_Case1	6952	11184
Matmul_Case0	4160	7104
Matmul_Case1	30976	55040

2.2 目标的耦合关系

缓存驻留峰值低，并不意味着每种缓存都能分配成功。一方面，不同类型不能互相借用空间；另一方面，分散的空闲区间无法容纳较大的连续申请。与此相反，过度追求小峰值可能使原本可重叠的计算串行化。因此首先通过生命周期收缩建立基准，再分别围绕搬运量和时间评价候选。

采用“拓扑顺序—连续分配—依赖重建—流水评价”的计算链。求解程序只产生候选，随后由不调用求解模块的重放程序从文本附件读回结果，重新检查空间和时间约束。两个程序共享输入字段定义，但不共享区间分配逻辑，减少同一实现错误同时影响生成和检查的可能性。

3. 模型假设与歧义处理

缓存大小沿用题面的抽象容量单位，不将其转换成字节；执行时间统一使用时钟周期。缓存申请与释放耗时为零，操作耗时固定为节点的周期字段。原始依赖边均不可删除，也不依据真实芯片的额外机制添加题面没有给出的吞吐假设。

问题一的驻留量按所有缓存类型合计计算，而非各类型峰值的最大值。对同一类型同时驻留的缓冲区要求地址不重叠，不同类型拥有独立地址空间。

附录 C 通过前一缓冲区的释放节点约束后一缓冲区的申请节点，直接描述了未发生换出的地址复用。发生换出时，物理地址在换出结束后即可复用，缓冲区的逻辑生命则持续到原始释放节点。本文将两者区分为逻辑生命周期与物理驻留段：申请或换入开启一段驻留，释放或换出结束该段。新驻留段必须等待所复用旧段的结束。若仍以最终释放节点约束换出期间的所有复用，可能人为形成依赖环，无法表达题面允许的换出机制。因此驻留段是本文对该情形的明确补充解释；与未提供的官方评测程序的一致性尚不能确证。

换出目标曾被任一输入复制节点引用时，按题面规定只计换入搬运量，换出时间为零；不自行判断数据是否被后续修改并改变计费方式。所有缓存类型均依题面统一换出规则处理。问题三要求额外搬运量不高于问题二选定方案，从而避免人为设定宽松的容忍阈值。

4. 符号说明

容量与搬运量均使用题面容量单位；时间及工作量使用时钟周期。节点编号、序列位置和地址偏移为整数。缓存类型集合记为 $\mathcal{T}$ 。本文用 F_v 表示结束时间，以免与边集 E 混淆。

表 2: 主要符号及取值

符号	定义与取值
$G = (V, E), V_o$	原始有向无环图、非管理操作节点集合
N, B, B_v	原始节点数、缓冲区集合、操作 v 引用的去重缓冲区集合
s_b, t_b, C_t	正整数缓冲区大小、类型及该类型容量
a_b, f_b	缓冲区 b 唯一的原始申请、释放节点
$\pi(v)$	节点位置；问题一为 V 到 $\{1, \dots, N\}$ 的双射
x_{bj}, o_{bj}, z_{bj}	第 j 个驻留段的非负整数偏移、开始与结束节点
χ_b	被任一 COPY_IN 引用时取 1，否则取 0
$\mathcal{M}, M$	换出事件集合及事件数；同一缓冲区可重复出现

表 2 (续)

符号	定义与取值
H, D, T	最大逻辑驻留量、额外搬运量、总执行周期
c_v, p_v	非负执行周期、指定执行单元；管理节点无独占单元
S_v, F_v, r_v	非负开始时间、结束时间、下游最长路径长度
L_H, L_0, L_R	同时使用容量下界、原始工作下界、固定驻留图时间下界
W, α	就绪窗口大小、启发式评分权重

5. 最小缓存驻留调度

5.1 生命周期模型与下界

问题一只优化节点顺序，不给缓冲区设置物理偏移，也不加入换入换出。缓冲区从申请后到释放前始终计入驻留量。对一个节点置换 π ，定义处理完序列第 k 个节点后的活跃指示量

$$y_b(k; \pi) = \mathbf{1}\{\pi(a_b) \leq k < \pi(f_b)\}, \quad k = 0, \dots, N. \quad (1)$$

于是调度模型为

$$\min_{\pi} H(\pi) = \max_{0 \leq k \leq N} \sum_{b \in B} s_b y_b(k; \pi), \quad (2)$$

$$\text{s.t. } \pi(u) < \pi(v), \quad (u, v) \in E, \quad (3)$$

$$\pi(a_b) < \pi(v) < \pi(f_b), \quad v \in V_o, b \in B_v, \quad (4)$$

$$\pi : V \longrightarrow \{1, \dots, N\} \text{ 为双射.}$$

式 (4) 把使用时必须活跃的语义显式列出；在输入依赖充分描述生命期时，它由原始路径蕴含。申请记正增量、释放记负增量的题面定义，与式 (2) 完全一致。总峰值是在同一位置对各类型求和后取最大值，不能相加各类型在不同位置达到的峰值。

每条操作的全部输入和输出缓冲区必须同时存在。因此，无论如何重排，都有

$$H^* \geq L_H := \max_{v \in V_o} \sum_{b \in B_v} s_b. \quad (5)$$

此界忽略跨操作必须保留的数据，计算成本低但可能较松。若某候选达到该界，可以证明其峰值最优；未达到时，只能据上下界判断尚未排除的改进空间。

5.2 固定操作顺序下的生命周期收缩

题面规定申请节点为根、释放节点为叶，六图均满足这一结构。固定非管理操作的相对顺序后，一个申请必须早于其所有显式后继以及所有引用该缓冲区的操作。称其中最早出现的节点为首次需求节点。相应地，释放必须晚于全部显式前驱和全部使用者，称其中

最晚出现的节点为末次需求节点。这里不能仅以第一次读或最后一次读代替需求节点，否则可能遗漏额外管理依赖。

收缩性质。在上述前提下，将申请移动到首次需求节点之前，将释放移动到末次需求节点之后，不会增加驻留峰值。

证明分两步。首先，申请是根节点，向后移动不会违反其入边；又因为尚未跨过任何显式后继或使用者，其出边和使用约束仍成立。将它与中间节点逐个交换，只会删除被跨越区间内这一缓冲区的正容量贡献。其次，释放是叶节点，向前移动不会违反其出边；停止在全部前驱和使用者之后，则其入边不受影响。逐个交换同样只会减少中间区间的驻留量。其他缓冲区的申请、释放相对关系不变，因此每一步交换的最大累计量均不增加。对所有缓冲区重复此变换，得到同一操作顺序下不劣的规范化序列。

这一性质消去了大量仅在管理节点位置上不同的候选，却没有解决操作顺序的组合搜索。特别是“尽早释放”要求末次使用已结束，并不允许提前释放随后还会用到的中间数据。若推广到申请有入边或释放有出边的图，必须重新限定可移动范围，不能直接使用本性质。

5.3 候选生成与算法对应

在操作诱导子图中维护入度和就绪集合。每选择一个操作，先插入尚未出现且为该操作所需的申请，随后发出操作，更新剩余使用次数；只有剩余使用次数为零且全部显式前驱已完成时才插入释放。输出完毕后逐边检查式 (3)，并检查节点置换及累计量首尾归零。

为了分开不同调度偏好的效果，设置三类主要操作顺序。编号顺序每次选择最小编号的就绪节点，是经过生命期收缩的强基准，而非未经处理的文件行顺序。缓存贪心先选就绪集合中编号最小的 W 个节点，再最小化

$$q_v = n_v - \alpha d_v, \quad n_v = \sum_{b \in B_v \setminus B_{\text{live}}} s_b, \quad d_v = \sum_{b \in B_v: \text{remain}(b)=1} s_b. \quad (6)$$

其中 d_v 是末次使用带来的释放潜力；若还有额外释放前驱未完成，实际释放仍须等待，评分不强制兑现这一潜力。相同评分按新增量、下游路径及编号打破平局。比较 $(W, \alpha) = (32, 1), (128, 1), (64, 2)$ ，窗口并非从所有就绪节点中寻找评分最好的 W 项。

第三类从编号有序的终端操作出发，沿前驱作深度优先访问，以完成访问顺序得到拓扑序；前驱正序和逆序各形成一个候选。它有机会缩短单个依赖分支的持续时间，但共享缓冲区可能使分支优先与复用优先发生冲突。另保留两种流水评分产生的操作顺序参与峰值比较。最终八种顺序均经过相同生命期收缩，再按真实峰值选优，不用局部评分直接替代目标函数。

令 $Q = \sum_{v \in V_o} |B_v|$, $K = \max_v |B_v|$ 。图遍历及未排序的深度优先核心为 $O(N + |E|)$ ；实现中的编号排序另有至多 $O((N + |E|) \log N)$ 开销。窗口贪心每步扫描就绪集合，并计算至多 W 个缓冲区评分，保守界为 $O(N^2 \log(W+1) + NWK + |E| + Q)$ 。空间为 $O(N + |E| + Q)$ 。实际并未枚举所有拓扑序，因此计算可承受性以限制搜索范围为代价。

5.4 六图峰值与解质量

表 3 同时给出收缩后编号基准、选定峰值及式 (5) 的下界。比值 H_1/L_H 是相对一个可计算下界的比值，不是已知的最优性误差。

表 3: 驻留峰值与同时使用下界（题面容量单位）

计算图	H_{id}	H_1	L_H	H_1/L_H
卷积小图	7560	6984	3144	2.22
卷积大图	14048	14048	932	15.07
注意力小图	4884	4884	512	9.54
注意力大图	7972	7972	512	15.57
矩阵乘小图	9728	9728	512	19.00
矩阵乘大图	35328	35328	512	69.00

卷积小图通过深度优先顺序从 7560 降至 6984，减少 576，降幅为 7.62%。其余五图在本轮候选中未降低编号基准的峰值；选定峰值分别为 14048、4884、7972、9728 和 35328。这一结果不能归结为“五图已最优”：例如矩阵乘大图的下界仅为 512，候选与下界相距甚远，既可能包含仍可消除的跨操作生命期，也反映同时使用下界本身过弱。

局部释放评分还可能明显恶化结果。卷积小图取 (32, 1) 时峰值为 21762，超过编号基准的两倍。评分只看到当前操作的新增量与末次使用潜力，没有估计后续分支累计的活跃集合。这一有限视野足以解释为什么不能把局部评分的下降视为全局峰值的下降；具体多出的活跃对象尚未逐个归因，不将这一解释当作已完成的因果分解。问题二仍需独立评价物理分配与搬运量，不能只凭本问峰值筛掉其他有价值的顺序。

6. 连续缓存分配与换入换出

6.1 逻辑生命与物理驻留状态

给定规范化顺序后，问题二还需要确定初始偏移、换出对象及其插入位置、配对换入位置和新偏移。问题允许进一步调整原始操作顺序，因此这些选择共同构成可行域。缓存容量分别为 $C_{L1} = 4096$ 、 $C_{UB} = 1024$ 、 $C_{L0A} = C_{L0B} = 256$ 、 $C_{L0C} = 512$ ，类型之间不能借用空间。

每个缓冲区在任一序列位置处于四种状态之一：未申请、驻留、已换出、已释放。合法状态迁移为

$$\text{未申请} \xrightarrow{\text{ALLOC}} \text{驻留} \begin{matrix} \xrightarrow{\text{SPILL_OUT}} \\ \xleftarrow{\text{SPILL_IN}} \end{matrix} \text{已换出}, \quad \text{驻留} \xrightarrow{\text{FREE}} \text{已释放}. \quad (7)$$

只有驻留状态允许操作引用；已释放后不可再次申请同一标识。换出不终结逻辑生命，仍须保留原始 FREE。若换出后已无普通操作需要该对象，也必须完成配对换入后再释放，以符合新增节点和配对规则，而不能删去换入以降低指标。

用序列区间 $[\pi(o_{bj}), \pi(z_{bj}))$ 表示第 j 段物理驻留, 从 ALLOC 或 SPILL_IN 开始, 到 FREE 或 SPILL_OUT 结束。其地址为整数半开区间 $[x_{bj}, x_{bj} + s_b)$, 满足

$$x_{bj} \in \mathbb{Z}_{\geq 0}, \quad x_{bj} + s_b \leq C_{t_b}. \quad (8)$$

若两个同类型驻留段在序列上重叠, 其地址必须满足析取约束

$$x_{bj} + s_b \leq x_{ak} \quad \text{或} \quad x_{ak} + s_a \leq x_{bj}. \quad (9)$$

每个使用者必须落在相应驻留段内, 段的关闭节点等待该段全部使用者, 下一次换入等待对应换出。时间上的地址互斥还由下一节的释放到再占用依赖保证; 仅在输出序列上画不重叠区间, 并不足以证明并行执行时安全。

6.2 搬运目标与连续空间的困难

令 χ_b 按题面 COPY_IN 引用关系取值。问题二的目标是

$$\min D = \sum_{(b,j) \in \mathcal{M}} (2 - \chi_b) s_b, \quad (10)$$

约束为原始节点拓扑顺序、式 (7)–(9) 及换出换入配对。重复换出同一对象按不同事件重复计费。一次换入耗时 $2s_b + 150$; 换出在 $\chi_b = 1$ 时耗时零, 否则也是 $2s_b + 150$ 。这些耗时进入问题三的执行单元工作量, 零耗时换出仍是依赖节点。

容量和连续性是两类约束。以容量 10 为例, 若两个当前需保留的对象位于 $[0, 2)$ 、 $[6, 8)$, 剩余空闲区间长 4 和 2; 新对象长 5, 三者总长仅为 9, 却没有一个空闲段可容纳它。因而“空闲总量足够”不是分配成功的充分条件。动态存储分配文献也将空闲区间组织与外部碎片作为核心问题 [2]; 本文使用最佳适配只是局部选择规则, 不据此断言碎片全局最少。

6.3 分配、换出与恢复算法

先沿原始顺序预存每个对象的未来使用位置队列, 再逐节点模拟四状态迁移。遇到申请或换入, 扫描对应缓存, 优先选择长度最小且足够的空闲区间。若没有可用区间, 排除当前操作必需的对象, 从其余同类型驻留对象中选择牺牲者, 插入换出并合并空闲区间, 重复直至分配成功。

牺牲评分比较两种选择: 最大化下一次使用距离 Δ_b , 或最大化

$$q_b^{\text{spill}} = \frac{\Delta_b}{(2 - \chi_b) s_b}. \quad (11)$$

无后续普通操作使用的对象, 将其下一次使用位置置于序列尾部之后。后一规则倾向于换出未来较晚使用且搬运较便宜的对象, 但没有计算不同对象被共同换出后形成连续空闲区间的组合收益。Bélády 对替换问题的研究利用完整未来引用序列构造离线比较基准 [3]; 本文只借鉴未来引用信息的作用。本题对象大小不等、一次操作需多个对象、存在连续地址限制, 故不能把经典分页的最优性结论移植给这两种评分。

普通换出可能被受保护对象的碎片阻塞, 此时启用显式恢复: 换出本次操作所需缓存类型中的全部驻留对象, 再按对象大小递减顺序重装本次必需集合。该步骤允许改变偏移,

但每次搬运均计入式 (10)，不是零成本地址搬移。当前实现以容量单元数组表示占用情况，便于检查边界；这也是容量不大时选择简单实现的原因。

恢复成功的前提为每个操作在每种缓存上的必需总量满足

$$\sum_{b \in B_v: t_b=t} s_b \leq C_t, \quad v \in V_o, t \in \mathcal{T}. \quad (12)$$

这一前提已对六图逐操作检查。证明如下：将相关类型清空后，在同一类型中把必需对象紧邻排列，其最后一个地址不超过总长，而总长不超过容量，所以存在连续放置。未换入的其他对象可留在 DDR 等待后续使用；DDR 容量在题面中不作限制。在固定原始顺序下，每个节点均可通过有限次换出和重装推进，因而不会因外部碎片永久停滞。若式 (12) 失败，单纯换出策略无法解决同时使用需求，应直接报告不可行，而不是无限循环。

初始偏移对应原始 ALLOC，每次换出另记录新的换入偏移。若有 M 次换出，新增节点恰为 $2M$ 个，第 j 次从零编号时使用 $N + 2j, N + 2j + 1$ ，并保持原始节点各出现一次。未来使用队列的预处理为 $O(N + Q)$ 。计入容量扫描、牺牲对象扫描及恢复时的对象排序，分配阶段的保守时间界为 $O((N + M)(C + |B| + K \log(K + 1)) + Q)$ ，其中 $C = \max_t C_t$ ；显式恢复增加的工作已包含在实际 M 内，不将其当作常数开销。

6.4 搬运量结果及策略取舍

问题二在编号、三组缓存贪心及两种深度优先顺序上，各比较两种换出评分，共十二个候选。按 D 最小选择，平局取较小执行时间。表 4 的编号基准采用相同生命周期收缩、最佳适配和距离评分，使比较反映顺序与牺牲规则的组合变化，而不是额外预处理差异。

表 4: 问题二的额外搬运量与换出次数

计算图	D_{id}	D_2	降幅/%	换出次数
卷积小图	87612	44766	48.90	56
卷积大图	76636	75546	1.42	2116
注意力小图	3620	3620	0.00	23
注意力大图	32852	32852	0.00	212
矩阵乘小图	34944	34944	0.00	273
矩阵乘大图	460800	460800	0.00	3600

卷积小图的搬运量从 87612 降至 44766，降幅 48.90%；顺序改为深度优先、仍用距离评分时已经降至 45336，再换用代价评分降至 44766。因此主要收益来自该图上的顺序变化，而代价评分进一步节省 570。卷积大图则选择深度优先与距离评分，搬运量为 75546；若对同一深度优先顺序改用代价评分，搬运量反而升至 81538。这一对照直接说明式 (11) 不是普遍优于距离评分的准则。

两个注意力图和两个矩阵乘图均保留编号基准的搬运量。尤其矩阵乘大图仍需 3600 次换出、搬运 460800 容量单位；不能从这一大数值推断它就是硬件容量造成的不可避免开销，

因为本文没有给出该实例的正搬运量下界。所有结果只表示已比较候选中的较优可行解。重复换出次数和搬运总量共同进入下一问，不应只比较峰值或只比较换出次数。

7. 搬运量约束下的流水优化

7.1 联合目标与时间递推

在有限缓存下，串行输出一个合法节点序列不等于硬件串行执行全部操作。同一执行单元的指令互斥，不同单元则可以并行；能否重叠取决于数据和地址依赖。以问题二的搬运量 D_2 为预算，问题三采用

$$\min T, \quad D \leq D_2, \quad T \geq F_v (v \in V_R), \quad (13)$$

并保留问题二的全部拓扑、驻留和地址约束。零增量预算是对题面“不显著增加”的保守具体化，不假设题面存在某个未给出的百分比阈值。

构造驻留依赖图 $G_R = (V_R, E_R)$ 。除原始边外，段开始节点指向该段使用者，全部使用者指向段结束节点，换出指向配对换入。逐地址单元跟踪上次结束驻留的节点；再次占用时，添加从旧段结束到新段开始的边。若连续多个段使用同一地址，只需连接相邻段，前后关系可由路径传递。该做法也覆盖部分区间相交，不能仅比较起始偏移是否相等。

给定 G_R 的一个拓扑序，记同一单元上序列中紧邻 v 的前一操作为 $h(v)$ ，则

$$S_v = \max \left\{ 0, \max_{u: (u,v) \in E_R} F_u, F_{h(v)} \right\}, \quad (14)$$

$$F_v = S_v + c_v, \quad T = \max_{v \in V_R} F_v. \quad (15)$$

无相应前项时取零；管理节点不加入执行单元互斥队列。式 (14) 取所有必要等待的最大值，所以同时满足非负、依赖和单元互斥。按拓扑序归纳，任何保持相同各单元指令顺序的可行时间表，都不能把当前节点提前于该最大值。因此递推得到的是该顺序下的最早执行时间表，而不是所有顺序的最短时间表。

7.2 固定驻留图重排为何保持可行

只保留 E_R 中的结构依赖，暂不固定同一执行单元的相邻次序，就获得一个可改变流水顺序的搜索空间。对每个地址单元，原方案的驻留段由“旧段结束先于新段开始”连接成链；每个段的使用者又位于开启与关闭之间。任意新的 E_R 拓扑序均保留这两种关系，所以既不会把使用者移动到段外，也不会让同一地址的两个段交叠。不同地址允许交换次序，不影响空间互斥。随后按式 (14) 加入新的单元顺序，时间上仍然可行。

这里固定的是对象、偏移及各地址上的驻留先后关系，而不是原序列中所有节点的总顺序。换出事件的对象和次数不变，式 (10) 的每一项随之不变，故固定驻留图重排严格保持 D 。这一性质比事后观察搬运量相同更强，但只适用于固定事件集合的阶段；地址重分配本身仍须单独接受预算检查。

7.3 候选扩展与列表调度

首先扩展原始候选：增加两种按估计最早开始时间与新增容量加权的操作排序，窗口均为 64，新增容量权重分别为 0、1 周期/容量单位；对八种顺序的距离换出方案分别尝试地址轮换。轮换从上次分配末端继续查找空闲区间，以改变地址复用形成的依赖。它同时可能改变碎片和换出次数，因而不能声称轮换天然保持搬运不变。初始候选共二十四四个，仅保留满足式 (13) 者，从中取得时间最小方案。

其次分别以问题二方案及扩展候选的最佳时间方案为起点，构造固定驻留图。按逆拓扑序计算

$$r_v = c_v + \max_{u:(v,u) \in E_R} r_u, \quad (16)$$

无后继时最大项取零。每个执行单元维护以 r_v 降序排列的就绪队列，管理节点就绪后立即处理。比较两种选择：其一在各队首中选择估计可开始时间最早者，同分优先较大的 r_v ；其二直接优先较大的 r_v 。新顺序必须再次通过完整依赖与地址重放，只有真实 T 下降才替换原方案。

采用下游路径排序的动机与 HEFT 的向上秩优先思想相关 [4]，但本文每个节点的执行单元已固定，不做 HEFT 的异构处理器分配，也没有复现其插空调度过程。式 (16) 使用本题固定时长及驻留依赖，而不是处理器平均时长与跨处理器通信代价，不能直接套用文献中的性能结论。

设 $N' = N + 2M$ 。路径计算与边遍历为 $O(N' + |E_R|)$ ，优先队列维护为 $O(N' \log N')$ ；固定数量单元的队首比较为 $O(N')$ 。若计入逐单元地址依赖构造，还需 $O(N'C)$ 的保守扫描开销。该策略没有回溯改变已保留的地址链，所以算法高效但搜索受限。

7.4 分阶段结果与改善来源

表 5 将不同阶段分开。 T_A 为扩展初始候选后的最佳时间； T_F 为仅从问题二方案出发固定驻留图重排的最佳时间； T_3 在两种起点、两种排序规则与已有方案间选优。 T_A 与 T_F 是两条可比较支路，不是按表列顺序连续执行的三个步骤。六图最终 $D_3 = D_2$ ，搬运量对应表 4。

表 5: 预算不增加条件下的时间对照（周期）

计算图	T_2	T_A	T_F	T_3	降幅/%
卷积小图	472038	472038	465892	465892	1.30
卷积大图	1210619	1210619	1044075	1044075	13.76
注意力小图	47509	47509	37744	37744	20.55
注意力大图	225902	225902	150499	150499	33.38
矩阵乘小图	194331	177625	186969	173274	10.84
矩阵乘大图	1800001	1765989	1758881	1749773	2.79

卷积与注意力四图的 $T_A = T_2$ ，而 $T_F = T_3 < T_2$ ，所以已取得的改善可以归于固定驻留关系后的重排。矩阵乘小图则先由扩展候选从 194331 降至 177625，再降至 173274；仅重排问题二方案只能达到 186969。矩阵乘大图也呈现相同次序，说明在这两个实例中，固定原地址关系会限制进一步收益。这里的“收益来源”限于已运行的两条支路比较，并非所有策略组合的完全因子试验。

为直观比较相对改善，图 1 统一以各图自己的 T_2 为分母，纵轴为 $100(T_2 - T)/T_2$ 。图中只比较同一时间指标的降幅，不暗示六图的绝对工作量相等。

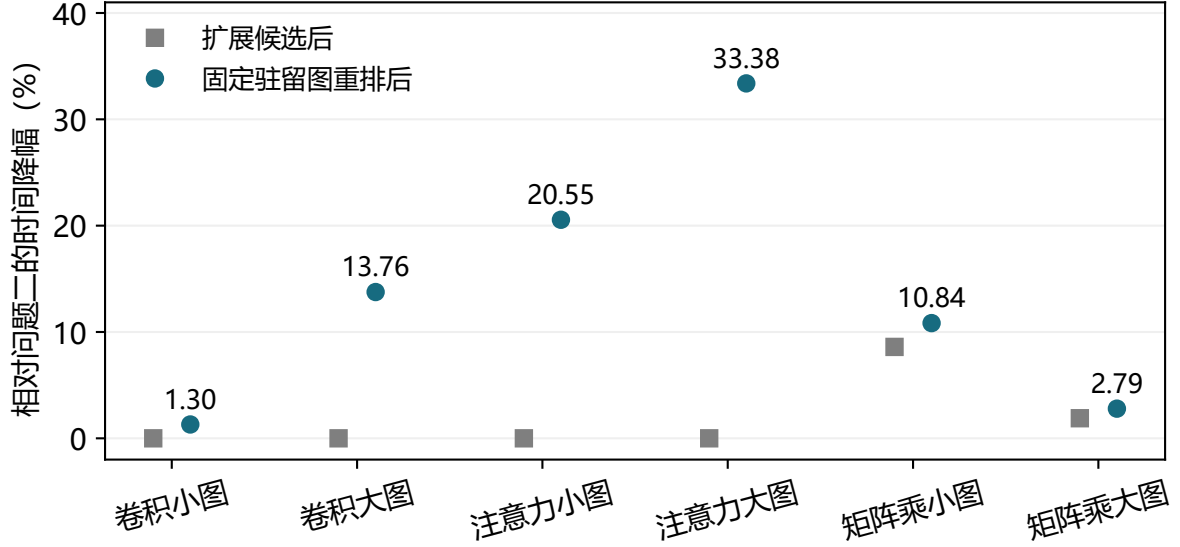


图 1: 扩展候选与最终重排的时间改善

注意力大图降幅 33.38% 最大；矩阵乘大图只有 2.79%。单靠降幅不能判断哪一图“调度更差”：大图的工作量下界与剩余依赖结构不同，需要进一步与可计算下界比较。

7.5 分层下界及瓶颈解释

令 $P(G)$ 为原始图按节点时长计算的最长路径， W_p^0 为单元 p 的原始工作量。忽略全部新增搬运和地址限制，得到对联合问题也成立的弱下界

$$L_0 = \max\{P(G), \max_p W_p^0\}. \quad (17)$$

固定换出事件后，记包括新增操作的工作量为 W_p ，则 $L_{\text{work}} = \max\{P(G), \max_p W_p\}$ 是该事件集合下的下界。进一步固定驻留图，令 $P(G_R)$ 不包含候选单元排序边的最长路径，得到

$$L_R = \max\{P(G_R), \max_p W_p\}, \quad g_R = 100(T_3/L_R - 1)\%. \quad (18)$$

证明只需注意两点：任一依赖路径必须串行完成，同一单元的全部工作也不能相互重叠。取二者最大值不超过任何合法总时间。加入已选单元排序边虽会得到更长路径，却把搜索决策固定成现有答案，不能再用来证明其他单元顺序也必须如此缓慢。

表 6 列出三个层次，尤其要避免把 g_R 当作联合问题的全局最优性差距。更换换出集合或地址关系时， L_{work} 和 L_R 也会变化。

表 6: 最终方案的分层时间下界及固定驻留差距

计算图	L_0	L_{work}	L_R	T_3	$g_R/\%$
卷积小图	348677	348677	458852	465892	1.53
卷积大图	409566	761650	835577	1044075	24.95
注意力小图	25600	25600	31953	37744	18.12
注意力大图	104960	104960	127789	150499	17.77
矩阵乘小图	65536	152438	152438	173274	13.67
矩阵乘大图	524288	1628000	1628000	1749773	7.48

卷积小图的原始工作下界为 348677，但固定驻留路径使下界提高到 458852，最终 465892 仅高出 7040 周期。由此只能判断保留当前驻留关系后的重排空间较小，不能判断地址方案接近全局最优。卷积大图相对固定驻留下界仍有 24.95% 的差距，下界尚不能排除较大的改善空间；但多资源约束可能使下界无法达到，故这一差距也不等于可实现的时间降幅。

表 7 进一步用最大工作量单元的忙碌比例解释不同图的表现。此处 W_p 在问题二和最终方案中相同；忙碌比例把执行窗口中的非工作部分计为等待，并不区分数据、地址和调度顺序分别贡献了多少等待。

表 7: 最大工作量单元的忙碌比例

计算图	最大工作量单元	W_p	$W_p/T_2/\%$	$W_p/T_3/\%$
卷积小图	CUBE	348677	73.87	74.84
卷积大图	MTE2	761650	62.91	72.95
注意力小图	VECTOR	25600	53.88	67.83
注意力大图	VECTOR	104960	46.46	69.74
矩阵乘小图	MTE2	152438	78.44	87.98
矩阵乘大图	MTE2	1628000	90.44	93.04

注意力大图的向量单元从 46.46% 提升至 69.74%，工作量仍为 104960 周期，改善确实来自总时间窗口的收缩。矩阵乘大图的输入搬运单元原已达到 90.44%，最终达到 93.04%；其 1628000 周期的工作量形成较强限制，因此 2.79% 的降幅与“忙碌单元剩余空隙较少”一致，但没有证明其他搬运方案无法降低工作量。

为观察等待分布，图 2 展示注意力小图在相同横轴上的两份完整流水。细条表示实际操作区间，空白表示该单元未执行操作；换出为零周期时不画成有宽度的条块，但其依赖仍参与计算。

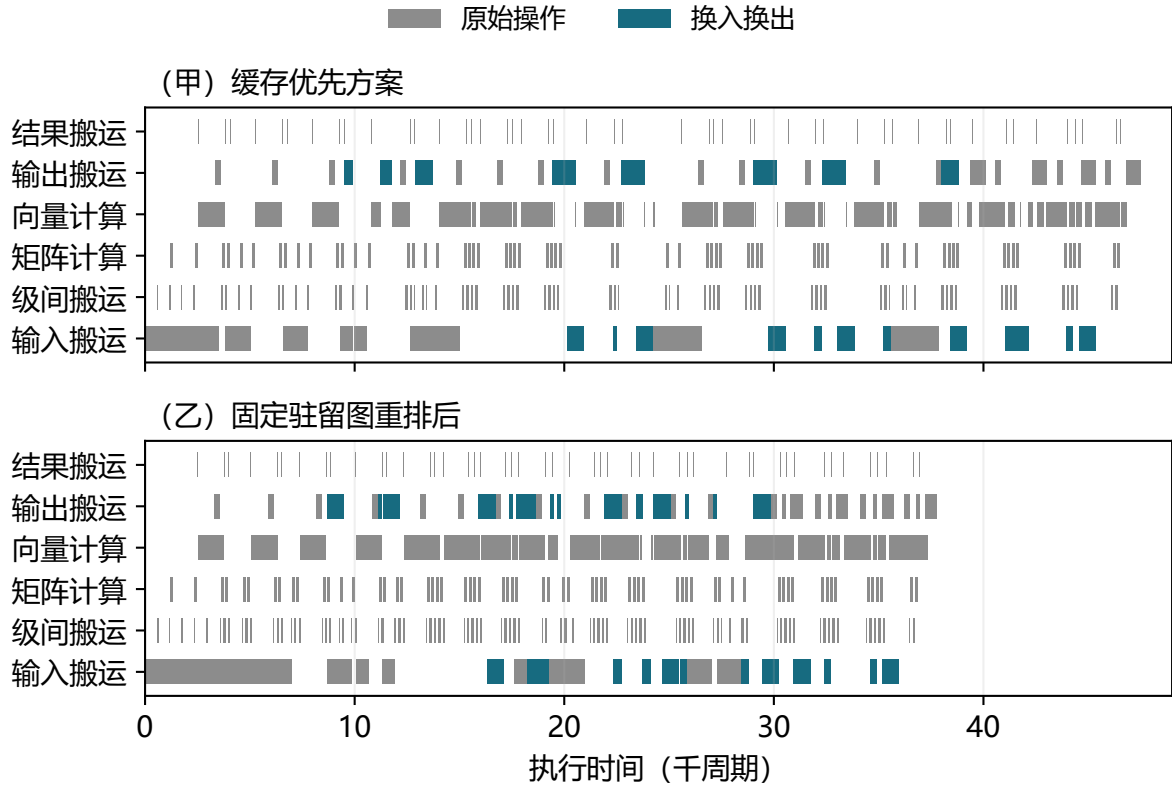


图 2: 注意力小图的优化前后流水 (同一时间尺度)

两方案搬运量同为 3620, 总时间从 47509 降至 37744 周期, 向量单元忙碌比例从 53.88% 升至 67.83%。图的作用是展示完成窗口与跨单元重叠, 不将单个微小条块的可辨性作为指标证据; 精确时间与利用率以表 5、表 7 为准。

8. 敏感性、边界与验证

8.1 窗口参数的可辨识性

候选规则全部一致应用于六个图, 不按算子名称选用专门参数。为分离窗口影响, 表 8 只比较释放权重 $\alpha = 1$ 、距离换出、最佳适配都固定时的 $W = 32$ 和 $W = 128$ 。 H_W 是原始序列峰值, D_W 是随后分配的额外搬运量。

表 8: 固定其他规则时的窗口对照

计算图	H_{32}	H_{128}	D_{32}	D_{128}	顺序相同
卷积小图	21762	21762	95564	95564	否
卷积大图	157630	180030	407112	439760	否
注意力小图	5536	5536	24896	24896	是
注意力大图	19008	19008	129512	129512	否
矩阵乘小图	14208	24704	85632	152832	否
矩阵乘大图	39680	52352	611968	719488	否

矩阵乘小图扩窗后，峰值从 14208 升至 24704，搬运量从 85632 升至 152832，说明扩大就绪选择范围不会单调改善结果。窗口改变的是每一步的可选集合，不是对已完成全局方案作包含关系扩展；一旦局部选择改变，后续状态也改变，所以“更多选择”不保证最终更优。

卷积小图与注意力大图两种窗口的峰值和搬运量相同，但完整序列并不相同，时间分别为 642017 与 640953、485807 与 486988 周期。这是目标统计量无法区分某些顺序差异的例子，不能据此认定参数未生效。注意力小图的两份完整序列确实相同，只能认定这两个窗口值在该实例上落在相同输出区间；本轮未记录每次候选竞争，尚不能区分窗口未约束就绪集合与最优局部选择恰好一致这两种原因。

此外，(64, 2) 同时改变窗口和权重，因此只作为另一个算法候选，不把其与 (32, 1) 的差异单独归因于释放权重。本轮未进行容量扰动实验：改变容量意味着改变题目边界条件，现有结果不足以支持对任意缓存规模的稳健性结论。

8.2 手算对照与错误注入

可行性检查从导出的序列、偏移与换出列表重新读入，不调用候选生成或分配函数。它检查原始及新增节点恰好出现一次、原始边顺序、换出配对、每次地址申请的整数边界与逐单元占用、操作使用时的驻留状态，最后重新计算流水和搬运量。问题三与问题二的十二份完整方案及六份问题一序列均完成相应重算。

为了防止执行时间计算把所有操作错误串行化，构造两个无依赖操作，分别在输入、输出搬运单元执行 10 和 20 周期，缓冲区大小均为 4。地址互不相交时，正确总时间为 $\max(10, 20) = 20$ ；复用同一地址时，前段释放到后段申请形成依赖，正确总时间为 $10 + 20 = 30$ 。这两个答案只差地址关系，可同时检验并行能力和地址等待是否生效。

对大小为 4 的输入复制缓冲区，一次换出搬运量为 4，换入耗时为 $2 \times 4 + 150 = 158$ ；按测试给定的其他操作和依赖，完整总时间为 188。对应非输入复制对象，搬运量为 8，两次搬运各耗时 158，完整时间为 346。这些手算测试覆盖零耗时换出和双向计费。再分别注入地址重叠、拓扑逆序、越界偏移及节点重复，检查程序均拒绝对应附件。拒绝非法输入只是验证的一部分，不替代对合法实例的数学建模判断。

8.3 解释边界与尚未解决的问题

本题附录 C 的地址复用描述直接针对 ALLOC/FREE；本文将其延伸为驻留段结束到开始的依赖。若把已换出的缓冲区仍视为占用地址直到最终 FREE，就无法表达换出释放空间的目的，并可能生成环。因此采用驻留段解释有物理上的一致性，但尚无官方评价器作交叉确认。相关时间结论均以此解释为前提。

对搬运量，本文只有非负这一平凡下界，没有得到六图的最小必要换出集合；对驻留峰值，式 (5) 也较弱。这使模型能证明方案满足约束，却不能证明搬运和峰值接近全局最优。时间下界更有辨识力，但强下界仅针对固定事件或驻留图。三种解质量保证的范围不同，不应合并成一个“算法最优”结论。

六图都是题面示例，没有独立保留的泛化测试集；相同规则跨图应用，只证明实现没有写入算子专用分支。进一步的通用性评价需要不同规模和不同共享数据结构的新图。求解与重放虽分开实现，仍共享输入解析和题意解释，不能排除共同理解偏差。真实芯片测量也未进行，周期改善不等同于实测吞吐加速。

9. 模型评价与结论

本文围绕三个递进目标，分别把节点顺序、物理空间和流水等待显式化。生命周期收缩在固定操作顺序下不增峰值；连续分配通过完整驻留状态与有代价的恢复机制获得可行方案；固定驻留图重排保持地址先后和搬运事件不变，从而将时间优化转化为受资源约束的拓扑排序。

六图中，问题一只有卷积小图相对收缩后的编号基准降低峰值，降幅为 7.62%。问题二的搬运量依次为 44766、75546、3620、32852、34944 和 460800。问题三在这些搬运量均不增加的条件下，将总时间降低 1.30% 至 33.38%；其中注意力大图从 225902 降至 150499 周期。阶段比较表明，四图的改善来自固定驻留重排，两个矩阵乘图还需要先改变地址分配候选。

这些结果支持“缓存方案确定后仍存在可利用的流水优化空间”，但没有证明当前搬运量或峰值最优。卷积小图接近固定驻留下界，进一步工作应考虑改变驻留关系；卷积大图与该下界仍相差 24.95%，值得扩展单元顺序搜索。矩阵乘大图则受到输入搬运工作量的明显限制，单纯在固定事件集合内重排的空间较小。后续改进方向应由这些不同证据决定，而不是对所有图统一增加启发式复杂度。

参考文献

- [1] 中国研究生数学建模竞赛组织委员会. 2025 年中国研究生数学建模竞赛 A 题：通用神经网络处理器下的核内调度问题及附件. 2025.
- [2] Wilson P R, Johnstone M S, Neely M, Boles D. Dynamic Storage Allocation: A Survey and Critical Review. International Workshop on Memory Management, LNCS 986, 1995: 1–116. DOI: 10.1007/3-540-60368-9_19.

- [3] Bélády L A. A study of replacement algorithms for a virtual-storage computer. IBM Systems Journal, 1966, 5(2): 78–101. DOI: 10.1147/sj.52.0078.
- [4] Topcuoglu H, Hariri S, Wu M Y. Performance-effective and low-complexity task scheduling for heterogeneous computing. IEEE Transactions on Parallel and Distributed Systems, 2002, 13(3): 260–274. DOI: 10.1109/71.993206.
- [5] 中国研究生数学建模竞赛组织委员会. “华为杯”第二十二届中国研究生数学建模竞赛人工智能工具及输出使用规定. 2025. <https://cpipc.acge.org.cn/sysFile/downloadFile.do?fileId=e3edba52b87c4bf7922f737c35c14b74>. 访问日期: 2026-09-21.

A. 复算与人工智能使用说明

支撑材料给出三问的完整调度文本。问题一只含原始节点顺序；问题二、三分别提供扩展节点顺序、初始地址偏移与按换出顺序列出的新偏移。新增节点每对由换出及换入组成，重新排列后也保持列表与节点编号一致。复算应从原图和这些文本重新建立依赖、空间状态及执行时间，不能只读取汇总数值。

程序以 Python 实现。图读取、顺序生成、地址分配、流水重排与附件重放分模块组织；随机种子不适用，因为候选规则及平局处理均为确定性。候选保留失败记录，论文表中的最终值来自同一组六图方案。本研究不把程序正常结束或结果文件存在当作数值正确的充分条件。

模型推导、程序及文字分析使用 OpenAI Codex 辅助。工具精确模型标识和版本发布日期未核实，参赛队员独立复核尚未完成。本阅读稿不构成人工原创、独立审查或正式提交资格的声明。若用于参赛，应由队员依据适用规定理解、核验并完成真实工具记录与使用声明 [5]。